Cheatsheet de Git y Github

Guías de Git

A continuación, encontrarás 3 guías que te ayudarán a comprender mejor el uso de Git y sus convenciones. Las guías están en este archivo para que las tengas a la mano.

Como crear ramas

https://codegym.cc/es/groups/posts/es.454.trabajo-en-equipo-sin-confusion-comprender-las-estrategias-de-ramificacion-en-git

---

Tutorial completo de Git

https://www.atlassian.com/es/git

Conventional Commits

https://www.conventionalcommits.org/es/v1.0.0/

Descarga de Sourcetree (herramienta visual para Git)

https://www.sourcetreeapp.com/

Resumen

Esta guía rápida te ayudará a trabajar con Git: Tienes la información sobre cómo nombrar ramas, escribir mensajes de commit efectivos y entender el flujo de trabajo estándar en proyectos colaborativos.

Prefijos de Ramas (Branch Naming)

Usamos una convención clara y estándar para organizar nuestro trabajo.

Convenciones principales

Prefijo	Uso
`feature/`	Nuevas funcionalidades o mejoras
`bugfix/`	Corrección de errores detectados.
`hotfix/`	Arreglos urgentes en producción
`release/`	Preparación de una nueva versión estable
`develop`	Rama base para desarrollo
`main` / `master`	Rama principal estable del proyecto.
`test/`	Cambios experimentales o pruebas.
`docs/`	Cambios exclusivos en la documentación.
`chore/`	Mantenimiento, tareas sin impacto funcional directo.

Ejemplos:

- `feature/user-registration`
- `bugfix/fix-login-bug`

- `hotfix/crash-on-startup`
- `release/v1.2.0`
- `docs/update-readme`
- `chore/update-dependencies`

Prefijos para Mensajes de Commit

Un buen mensaje de commit facilita la lectura del historial del proyecto. Debe ser breve, claro y estar en **inglés**.

Lista de prefijos válidos

Prefijo	Uso
`feat:`	Añade una nueva funcionalidad.
`fix:`	Corrige un error.
`perf:`	Mejora de rendimiento.
`chore:`	Cambios menores de mantenimiento o configuración.
`build:`	Cambios en el sistema de build o scripts de despliegue.
`ci:`	Cambios en la integración continua.
`docs:`	Cambios en la documentación.
`refactor:`	Reestructuración de código sin cambiar su funcionalidad.
`style:`	Cambios de formato o estilo sin impacto en la lógica.
`test:`	Añadir o mejorar pruebas automáticas.

Reglas para buenos commits

1. **Siempre en inglés.**
2. **Siempre en imperativo.** Ej: `add`, `fix`, `refactor`, no `added`, `fixed`.
3. **Usa un scope entre paréntesis** para indicar la parte afectada del código.
4. **Incluye un mensaje claro y descriptivo.**

✍ Ejemplos de Commits (Español e Inglés)

> Usa estos ejemplos como referencia para tus propios commits.

`feat` – Nueva funcionalidad

Español:

```
feat(auth): agregar funcionalidad de registro de usuarios
feat(profile): implementar edición de biografía de usuario
feat(dashboard): agregar widget de análisis
feat(settings): implementar alternancia de modo oscuro
```

English:

```
feat(auth): add user registration functionality
feat(profile): implement user bio editing
feat(dashboard): add analytics widget
feat(settings): implement dark mode toggle
```

🐛 `fix` – Corrección de errores

Español:

```
fix(auth): resolver problema con el restablecimiento de contraseña
fix(ui): corregir alineación de botones en la página principal
fix(dashboard): corregir renderizado incorrecto de datos en gráficos
fix(settings): resolver problema al guardar preferencias de usuario
```

English:

```
fix(auth): resolve password reset issue
fix(ui): fix button alignment on the homepage
fix(dashboard): fix incorrect data rendering in charts
fix(settings): resolve issue with saving user preferences
```

⚡ `perf` – Mejora de rendimiento

Español:

```
perf(api): optimizar consultas de búsqueda de usuarios
perf(images): comprimir imágenes para una carga más rápida
perf(dashboard): mejorar tiempo de carga para grandes conjuntos de datos
perf(settings): optimizar renderizado de la página de configuración
```

English:

```
perf(api): optimize user search queries
perf(images): compress images for faster loading
perf(dashboard): improve loading time for large datasets
perf(settings): optimize settings page rendering
```

🔧 `chore` – Mantenimiento y configuración

Español:

```
chore(dependencies): actualizar todos los paquetes npm a las últimas versiones
chore(config): agregar configuración de Prettier para formateo de código
chore(lint): agregar reglas de linting para estilo de código consistente
chore(env): actualizar variables de entorno para staging
```

English:

```
chore(dependencies): update all npm packages to latest versions
chore(config): add Prettier configuration for code formatting
chore(lint): add linting rules for consistent code style
chore(env): update environment variables for staging
```

🏗 `build` – Sistema de compilación y despliegue

Español:

```
build(ci): configurar Webpack para compilaciones de producción
build(scripts): agregar script para migraciones de base de datos
build(docker): agregar Dockerfile de múltiples etapas
build(deploy): configurar script de despliegue para AWS
```

English:

```
build(ci): configure Webpack for production builds
build(scripts): add script for database migrations
build(docker): add multi-stage Dockerfile
build(deploy): configure deployment script for AWS
```

👷 `ci` – Integración Continua

Español:

```
ci(pipeline): corregir pipeline de CI para ramas de características
ci(deployment): agregar mecanismo de reversión al flujo de trabajo CI/CD
ci(testing): agregar pruebas automatizadas al pipeline de CI
ci(build): corregir paso de compilación en el flujo de trabajo de CI
```

English:

```
ci(pipeline): fix CI pipeline for feature branches
ci(deployment): add rollback mechanism to CI/CD workflow
ci(testing): add automated tests to CI pipeline
ci(build): fix build step in CI workflow
```

📚 `docs` – Documentación

Español:

```
docs(readme): actualizar instrucciones de configuración del proyecto
docs(api): agregar ejemplos de uso para los endpoints de la API
docs(contributing): agregar guía de contribución
docs(changelog): actualizar changelog para la última versión
```

English:

```
docs(readme): update project setup instructions
docs(api): add usage examples for API endpoints
docs(contributing): add contributing guidelines
docs(changelog): update changelog for latest release
```

♻️ `refactor` – Mejora de estructura del código

Español:

```
refactor(auth): simplificar middleware de autenticación
refactor(ui): reestructurar componentes para mejor reutilización
refactor(routes): limpiar definiciones de rutas
refactor(services): dividir servicios en módulos más pequeños
```

English:

```
refactor(auth): simplify authentication middleware
refactor(ui): restructure components for better reusability
refactor(routes): clean up route definitions
refactor(services): split services into smaller modules
```

💄 `style` – Formato y estilo

Español:

```
style(css): corregir espaciado inconsistente en hojas de estilo
style(js): eliminar variables no utilizadas en archivos JavaScript
style(html): corregir la indentación en archivos HTML
style(js): formatear código con Prettier
```

English:

```
style(css): fix inconsistent spacing in stylesheets
style(js): remove unused variables in JavaScript files
style(html): fix indentation in HTML files
style(js): format code with Prettier
```

`test` – Pruebas

Español:

```
test(auth): agregar pruebas de integración para el flujo de inicio de sesión
test(api): refactorizar pruebas para usar datos simulados
test(routes): agregar pruebas unitarias para manejadores de rutas
test(services): mejorar cobertura de pruebas para la capa de servicios
```

English:

```
test(auth): add integration tests for login flow
test(api): refactor tests to use mock data
test(routes): add unit tests for route handlers
test(services): improve test coverage for service layer
```

¿Cómo estructurar un Pull Request (PR)?

Cuando trabajes en equipo o subas cambios a un repositorio compartido, deberás crear un **Pull Request**. Esta es una solicitud para que tus cambios se integren a la rama principal.

Buenas prácticas para escribir un PR

- **Título** claro y conciso, similar al primer commit.
- **Descripción** explicando el *qué* y el *por qué* de los cambios.
- Divide en secciones:
  - Qué se ha añadido o corregido
  - Qué se ha probado y cómo
  - Relación con tareas/tickets si los hay
- Escribe en **inglés**, al igual que los commits.

Ejemplo de descripción de PR

Español:

```
### ¿Qué se ha hecho?
- Se añadió el formulario de registro con validaciones
- Se creó un hook personalizado para la lógica de autenticación

### Correcciones
- Se solucionó el bug que mantenía la sesión iniciada tras logout
- Se corrigió un warning de input no controlado en el login

### Tests
- Se probó manualmente el flujo de registro e inicio de sesión
- Se añadieron pruebas unitarias para el reducer de autenticación

### Relacionado
Closes #42 (esto ya lo veremos más adelante)
```

English:

```
### What’s new?
- Added registration form with input validation
- Created custom hook for authentication logic

### Fixes
- Fixed bug where user remained logged in after logout
- Resolved warning about uncontrolled input in login form

### Tests
- Manually tested registration and login
- Added unit tests for auth reducer

### Related
Closes #42 (esto ya lo veremos más adelante)
```

Tabla Resumen

Tipo	Prefijo	¿Qué hace?
Funcionalidad nueva	`feat:`	Añade una nueva función que no existía.
Corrección de errores	`fix:`	Soluciona un bug.
Rendimiento	`perf:`	Mejora velocidad o consumo de recursos.
Estilo o formato	`style:`	Cambios estéticos, sin impacto funcional.
Refactorización	`refactor:`	Mejora del código sin alterar su comportamiento.
Documentación	`docs:`	Cambios en documentación o comentarios.
Configuración	`chore:`	Tareas del proyecto, mantenimiento, dependencias.
Sistema de compilación	`build:`	Cambios en Webpack, Docker, scripts de build o despliegue.
Integración Continua	`ci:`	Cambios en Github Actions, pipelines, tests automáticos, etc.
Pruebas	`test:`	Nuevos tests, mejora de cobertura, test mocks.

¡Ahora estás listo para contribuir como auténtico Samurái! ⚔